OPEN SOURCE · CASHLESS PARA EVENTOS

StuStaPay — Arquitetura Técnica

Sistema de pagamentos cashless com pulseira NFC, equivalente open source ao Imply Cashless

Licença: AGPL-3.0
Stack: Python · PostgreSQL · React · Android
Origem: StuStaPay (DE) · github.com/stustapay
Audiência: Apresentação técnica ao cliente

CAMADA 1 · CLIENTE

Participante do Evento

Pulseira NFC entregue na entrada identifica o participante em toda a operação

Pulseira NFC NTAG213
`13.56MHz · NXP`
UID único + saldo no backend. Tecido resistente para festivais.

Recarga de Saldo
`Balcão · Web · Self-service`
Participante carrega a pulseira em dinheiro, cartão ou PIX.

Reembolso
`Ao final do evento`
Saldo residual devolvido via balcão, site ou terminal self-service.

Approach NFC (1 toque · <1s)

CAMADA 2 · FRENTE DE CAIXA

PDV nas Barracas

Cada barraca tem um PDV autônomo que funciona 100% offline e sincroniza ao reconectar

Tablet / Celular Android
`Android 10+ · USB-OTG`
Samsung A9+, A54 ou equivalente. Tela de seleção de produtos.

Leitor ACR1222L
`USB · NTAG213/215`
Lê a pulseira em 50–80ms. Driver ACS nativo no Android.

App StuStaPay Android
`Kotlin · Compose · Hilt`
Interface de PDV: produtos, leitura NFC, registro de venda.

Cache Offline (SQLite)
`SQLite local + fila`
Transações em fila quando sem rede. Sync ao reconectar.

REST/JSON + WebSocket · Wi-Fi 5GHz dedicado

CAMADA 3 · APLICAÇÃO (SERVIDOR CENTRAL)

Backend + Painel Web

NUC/mini-PC no local do evento processa todas as transações em tempo real

Backend API
`Python 3.11 · FastAPI · asyncpg`
REST/GraphQL: PDV, recarga, reembolso, auth, gestão de produtos e taxas.

Painel Administrativo Web
`React 18 · TypeScript · Vite`
Configuração de evento, produtos, preços, usuários, relatórios.

Dashboards em Tempo Real
`WebSockets · Django Channels`
Vendas por barraca, saldo em circulação, transações/segundo, alertas.

Persistência + Cache + Fila

CAMADA 4 · DADOS & INFRA

Banco de Dados e Infraestrutura

Source of truth do evento. Backup automático e UPS garantem continuidade

PostgreSQL 15
`Schema completo · pg_dump`
Contas, transações, produtos, taxas, eventos, usuários.

Redis
`Cache + Fila de sync`
Cache de saldo e fila de reconciliação de transações offline.

Docker Compose
`1 host · NUC/Beelink`
Orquestra backend + DB + Redis + nginx em um único servidor.

UPS + Backup
`APC 1500VA · B2 Cloud`
Autonomia 30min + backup offsite a cada 15 min.

REDE NO EVENTO

- **Wi-Fi 5GHz dedicado** para PDVs (VLAN isolada)
- **Dual-WAN**: 4G + fixa com failover automático
- **QoS**: prioriza tráfego backend
- **Offline-first**: internet cair não derruba vendas

INTEGRAÇÕES

- **Pretix** — ingressos QR code
- **Stripe** — recarga online por cartão
- **PIX** — recarga via API banco (fase 2)
- **SAT/NF-e** — fiscal BR (fase 2, ~4–8 sem)

FLUXO DE TRANSAÇÃO

1. PDV lê pulseira (UID)
2. Consulta saldo no backend
3. Registra venda + débito
4. Atualiza dashboards em <1s
5. Offline → fila local → sync depois

VANTAGENS VS IMPLY

- **R$ 0** de licença (AGPL-3.0)
- **Dados no seu servidor** (soberania)
- **Hardware padrão aberto** (sem lock-in)
- **Código customizável**

<1s	100%	~50	R$ 0
LATÊNCIA POR TRANSAÇÃO	OPERAÇÃO OFFLINE	TX/SEG POR PDV	CUSTO DE LICENÇA
Leitura + consulta + registro	PDV continua vendendo sem rede	Capacidade comprovada em festivais DE	AGPL-3.0 · manter fork público